AI GEN X

Training & Architecting Autonomous Intelligence

Agentic Architecture & Framework Document

Batch 2 — Technical Deep-Dive: OpenAI Agents SDK + MCP Hybrid Architecture
for Dragonfruit Media's AI-Enabled Pipeline

BATCH 2 OF 6 — CONFIDENTIAL

Prepared for: Dragonfruit Media · Brendan, Yazan, Emily

Prepared by: AI Gen X · March 2026

Document Version: 2.0

Table of Contents

1 Architecture Philosophy & Design Principles
2 Grand Schema — Orchestration Layer Architecture
3 OpenAI Agents SDK Framework Deep-Dive
4 MCP vs. Function Tool Decision Framework
5 Agent Taxonomy — The Five Core Agents
6 Agent Detail: Producer Agent
7 Agent Detail: Production Coordinator (PC) Agent
8 Agent Detail: Creative Director (CD) Agent
9 Agent Detail: Post-Production Manager (PPM) Agent
10 Agent Detail: Editor Support Agent
11 Cross-Agent Handoff Architecture
12 Data Flow — End-to-End Request Lifecycle
13 Human-in-the-Loop (HITL) Gate Architecture
14 Security, Authentication & Error Handling
15 Infrastructure & Deployment Architecture
16 Demo Mode & Mock Data Architecture
17 Observability, Tracing & Monitoring

1. Architecture Philosophy & Design Principles

This document defines the complete agentic architecture for Dragonfruit Media's AI-enabled pipeline. Every design decision is grounded in DFM's specific workflow, time audit data, and the goal of scaling from 4 → 7 clients per pod by EOY 2026.

1.1 Core Design Principles

Principle	What It Means	Why It Matters for DFM
Pipeline-First	The DFM production pipeline (Onboarding → Pre-Pro → Production → Post-Pro → Publish) is the source of truth. Every agent maps to pipeline phases.	Ensures automation targets real workflow steps, not hypothetical ones. Directly addresses the 30% "Specificity to Workflow" evaluation criterion.
Role-Aligned Agents	One primary agent per core role (Producer, PC, CD, PPM, Editor). Each agent has role-specific instructions, tools, and permissions.	Matches DFM's pod structure. Producers and PCs share coordination load (74.5% and 70% overhead respectively). CDs need creative tools. PPMs need QC tools.
MCP-First, API-Fallback	Use official MCP servers where available (ClickUp, Slack, Notion). Build custom function tools for APIs without MCP (Frame.io, YouTube, Dropbox).	MCP servers are production-ready, maintained by platform vendors, and support OAuth. Reduces our maintenance burden for the 3 most-used integrations.
Human-in-the-Loop by Default	Every write operation (create task, send message, update status) requires human approval. Read operations are autonomous.	DFM's risk mitigation requirement: "Human-in-the-loop review; automated actions flagged in dedicated Slack channel." Zero tolerance for autonomous writes in Phase 1.
Async-First Communication	All agent interactions happen via Slack or dashboard. No synchronous dependencies. Results posted to channels.	Matches DFM's operating style: Slack-heavy, async-first, 22 countries, multiple time zones.
Observable & Traceable	Every agent run produces a trace: input → reasoning → tool calls → HITL decision → output. All traces viewable by DFM team.	Builds trust. Evaluators can see exactly what the agent did and why. Supports the "could we hand it to an engineer tomorrow" criterion.

1.2 Architecture Constraints (From DFM Brief)

> **From the Evaluation Criteria (Section 05):**
> • 30% — Specificity to Our Workflow: "Does it reference our pipeline, tools, and time

audit — not generic advice?"
• 25% — Clarity of Architecture: "Is the system design concrete? Could we hand it to an engineer tomorrow?"
• The architecture must be **engineer-ready**: connectors, function tools, agent definitions, API endpoints — all specified.

2. Grand Schema — Orchestration Layer Architecture

The grand schema shows how all components connect: user interfaces, the orchestration layer (OpenAI Agents SDK), MCP servers, custom connectors, external APIs, and HITL gates.

2.1 Architecture Layers

DRAGONFRUIT MEDIA — AGENTIC ARCHITECTURE (5 LAYERS)

LAYER 1 — USER INTERFACE

Slack (primary) · Dashboard (Streamlit/React) · CLI (dev/testing)

User sends natural language request → routed to appropriate agent

▼

LAYER 2 — ORCHESTRATION (OpenAI Agents SDK)

Runner.run(agent, message) → ReAct Loop (Reason → Act → Observe) → Tool Calls → Result

5 Role Agents: Producer · PC · CD · PPM · Editor Support

+ Orchestrator Agent (routes to correct role agent based on intent)

+ Handoff Protocol: Agent-as-Tool pattern for cross-role delegation

▼

LAYER 3 — TOOL LAYER (MCP Servers + Function Tools)

MCP Servers: ClickUp MCP (official) · Slack MCP (official) · Notion MCP (official)

Function Tools: Frame.io Connector · YouTube Connector · Dropbox Connector

AI Tools: Retention Scorer · QC Checker · Sentiment Analyzer · Capacity Engine

Each tool = Python function with typed parameters, exposed to agents via SDK

▼

LAYER 4 — HITL GATE LAYER

7 Critical Gates: Task Creation · Status Update · Message Send · Assignment · Review Routing · Invoice · Escalation

All write operations paused → Slack approval prompt → User confirms/rejects → Agent proceeds or aborts

▼

LAYER 5 — EXTERNAL SYSTEMS

ClickUp API · Slack API · Notion API · Frame.io API · YouTube Data API v3 · Dropbox API · Make/Zapier

Production systems — agents interact via Layer 3 connectors, never directly

2.2 Layer Interaction Rules

Rule	Description	Rationale
No Direct API Access	Agents never call external APIs directly. All access goes through Layer 3 (MCP or Function Tool).	Single point of control for auth, rate limiting, logging, and mock mode.
HITL Before Write	Every write operation (POST, PUT, DELETE) passes through Layer 4 before reaching Layer 5.	DFM requirement: "automated actions flagged in dedicated Slack channel."
Read = Autonomous	Read operations (GET) bypass Layer 4 and execute immediately.	Agents need real-time data to reason. Blocking reads would break the ReAct loop.
Orchestrator Routes	The Orchestrator Agent in Layer 2 determines which role agent handles the request based on intent classification.	Users don't need to know which agent to talk to. "Create a task" → Producer Agent. "Check QC" → PPM Agent.
Handoffs Are Explicit	When Agent A delegates to Agent B, it uses the SDK's handoff mechanism. The user sees the handoff in the trace.	Transparency. DFM leadership can see exactly when and why a Producer Agent handed off to a PC Agent.

3. OpenAI Agents SDK Framework Deep-Dive

The OpenAI Agents SDK is the orchestration framework for all DFM agents. This section explains why it was chosen, how it works, and how we configure it for DFM's specific needs.

3.1 Why OpenAI Agents SDK

Criteria	OpenAI Agents SDK	LangGraph	CrewAI / AutoGen	Raw MCP
Agent Definition	Agent = LLM + name + instructions + tools. Minimal boilerplate.	Nodes + edges + state graph. Heavy setup.	Role-based agents with task definitions. Medium setup.	No agent concept. Just tool servers.
Tool Binding	Python functions or MCP servers. Both first-class.	Python functions via tool nodes.	Python functions via decorators.	MCP only.
MCP Support	✅ Native. MCPServerStdio, MCPServerStreamableHTTP.	❌ Requires custom adapter.	❌ Requires custom adapter.	✅ Native (it IS MCP).
Handoffs	✅ Built-in. Agent-as-tool pattern.	Conditional edges (manual).	Task delegation (manual).	❌ No agent loop.
HITL / Guardrails	✅ Built-in guardrails + approval callbacks.	Human-in-the-loop via interrupt.	Human input via task config.	❌ No built-in.
Tracing	✅ Built-in tracing with OpenAI dashboard.	LangSmith integration.	Custom logging.	❌ No tracing.
Production Readiness	✅ Production-oriented. Used by OpenAI internally.	✅ Mature.	⚠️ Evolving.	⚠️ Protocol only.
Best For DFM	✅ **Lightest setup, native MCP, built-in handoffs, built-in HITL, built-in tracing.**	Overkill for our use case.	Too much orchestration overhead.	No agent reasoning loop.

3.2 SDK Core Concepts (Mapped to DFM)

Agent Definition

```
from agents import Agent, Runner, function_tool
from agents.mcp import MCPServerStdio

# Producer Agent — handles task creation, timeline management, client coordination
```

```
producer_agent = Agent(
    name="Producer Agent",
    instructions="""You are the Producer Agent for Dragonfruit Media.
    Your responsibilities:
    - Create and manage ClickUp tasks for pod batches
    - Route Slack messages to appropriate ClickUp tasks
    - Calculate project timelines based on content type and pod capacity
    - Monitor client sentiment across communication channels

    RULES:
    - Always confirm before creating/updating tasks (HITL gate)
    - Reference the pod structure: Producer, CD, PC, PPM + 10-12 Editors
    - Use DFM naming conventions: [Client] - [Content Type] - [Batch #]
    - Flag any task that would exceed pod capacity of 7 clients
    """,
    tools=[
        clickup_create_task,      # Function tool
        clickup_list_tasks,       # Function tool
        clickup_update_status,    # Function tool
        slack_send_message,       # Function tool
        timeline_calculator,      # Custom AI tool (PR-1)
    ],
    mcp_servers=[
        clickup_mcp_server,       # Official ClickUp MCP
        slack_mcp_server,         # Official Slack MCP
    ],
    handoffs=[pc_agent, cd_agent] # Can delegate to PC or CD
)
```

Runner Execution (ReAct Loop)

```
# The Runner executes the agent's ReAct loop:
# 1. REASON: Agent reads user message + instructions
# 2. ACT: Agent selects a tool and calls it
# 3. OBSERVE: Agent reads tool result
# 4. REPEAT until task is complete or HITL gate is reached

result = await Runner.run(
    agent=producer_agent,
    input="Create a new batch for Mango Pod: 4 longform videos for Client Alpha,
due March 21"
)

# Trace output (visible to DFM team):
# Step 1: Agent reasons → "Need to create 4 tasks in ClickUp for Mango Pod"
# Step 2: Agent calls clickup_list_lists(space="Mango Pod") → gets list IDs
# Step 3: Agent calls clickup_create_task(list_id=..., name="Client Alpha - LF -
Batch 12 - V1")
# Step 4: HITL GATE → Slack message: "Create task? [Approve] [Reject]"
# Step 5: User approves → task created → Agent confirms in Slack
```

3.3 Tool Types in the SDK

Tool Type	How It Works	Used For (DFM)	Example
Function Tool	Python function decorated with @function_tool. Agent calls it by name with typed parameters.	Custom logic, AI models, calculations, APIs without MCP	timeline_calculator, retention_scorer, qc_checker, frameio_get_comments
MCP Server Tool	External MCP server connected via MCPServerStdio or MCPServerStreamableHTTP. Tools auto-discovered.	ClickUp, Slack, Notion — official MCP servers with full API coverage	clickup_mcp.create_task, slack_mcp.send_message, notion_mcp.get_page
Agent-as-Tool (Handoff)	Another agent registered as a tool. Calling it delegates the conversation to that agent.	Cross-role delegation: Producer → PC, CD → PPM	producer_agent.handoffs = [pc_agent]

4. MCP vs. Function Tool Decision Framework

For every external system DFM uses, we make an explicit decision: use an official MCP server or build a custom function tool connector. This decision is based on MCP availability, API coverage, and maintenance burden.

4.1 MCP Availability Matrix (Confirmed via Research)

System	MCP Status	Decision	Rationale
ClickUp	✅ Official MCP	**Use MCP**	Official MCP server (public beta, all plans). Supports: create/update/delete tasks, lists, spaces, comments, assignees, statuses, custom fields. OAuth authentication. Community server (460+ GitHub stars) also available as fallback.
Slack	✅ Official MCP	**Use MCP**	Official Slack MCP Server (Streamable HTTP transport). Supports: search messages, send messages, read channels/threads, manage canvases, list users. Bot token authentication.
Notion	✅ Official MCP	**Use MCP**	Official Notion MCP Server (hosted, OAuth). Supports: get/create/update pages, search, databases, blocks. 4K+ GitHub stars. Used for SOPs, client knowledge bases, documentation.
Frame.io	❌ No MCP	**Custom Function Tool**	No official or community MCP server. Build custom connector using Frame.io API v2. Key operations: get_comments, get_assets, get_review_links. Used by PPMs and CDs for review workflows.
YouTube	❌ No MCP	**Custom Function Tool**	No official MCP. Build custom connector using YouTube Data API v3. Key operations: get_video_analytics, get_channel_stats, search_videos. Used by CDs for competitor research (CD-2).
Dropbox	❌ No MCP	**Custom Function Tool**	No official MCP. Build custom connector or use Make/Zapier integration. Key operations: list_files, download_file, get_shared_link. Used by PCs for footage management.
Make / Zapier	❌ No MCP	**Webhook Trigger**	Agents trigger Make scenarios via webhook URLs. Make handles multi-step automations that don't need agent reasoning (e.g., file move → notification → status update).
Adobe / Descript / ElevenLabs	❌ No MCP	**Indirect (via Make or manual)**	These are desktop/creative tools. Agents don't control them directly. Instead: agents prepare inputs

(scripts, briefs) and trigger downstream workflows via Make or Slack notifications.

4.2 Decision Tree

When to use MCP:
1. Official MCP server exists → Use it (ClickUp, Slack, Notion)
2. Community MCP server with 100+ stars and active maintenance → Evaluate, use if stable
3. We need broad API coverage (many operations) → MCP auto-discovers all tools

When to use Function Tools:
1. No MCP server exists → Build custom connector (Frame.io, YouTube, Dropbox)
2. We need only 2-3 specific operations → Function tool is lighter than full MCP
3. We need custom logic (AI models, calculations) → Always function tool
4. We need HITL gates on specific operations → Easier to implement in function tools

4.3 Hybrid Binding Example

```
# Producer Agent uses BOTH MCP and Function Tools simultaneously
producer_agent = Agent(
    name="Producer Agent",
    instructions="...",
    tools=[
        # Custom function tools (no MCP available)
        frameio_get_comments,        # Frame.io API v2
        timeline_calculator,         # Custom AI tool (PR-1)
        client_sentiment_tracker,    # Custom AI tool (PR-2)
    ],
    mcp_servers=[
        # Official MCP servers (auto-discover all tools)
        MCPServerStdio(
            name="ClickUp",
            command="npx",
            args=["@clickup/mcp-server"],
            env={"CLICKUP_API_KEY": os.getenv("CLICKUP_API_KEY")}
        ),
        MCPServerStreamableHTTP(
            name="Slack",
            url="https://mcp.slack.com/sse",
            headers={"Authorization": f"Bearer {os.getenv('SLACK_BOT_TOKEN')}"}
        ),
    ]
)
```

5. Agent Taxonomy — The Five Core Agents

Each agent maps to one of DFM's 5 core roles. The agent's instructions, tools, and permissions are calibrated to that role's specific workflow, time audit data, and dream allocation targets.

5.1 Agent Overview

Agent	Role	Primary Pipeline Phases	Key Tools	Wish List Coverage	Dream Allocation Target
Producer Agent	Producer	Onboarding, Pre-Pro, Post-Pro (coordination)	ClickUp MCP, Slack MCP, Timeline Calculator, Sentiment Tracker	PR-1, PR-2, PR-3, PR-4	Coordination 74.5% → 41%
PC Agent	Production Coordinator	Pre-Pro, Post-Pro, Publish	ClickUp MCP, Slack MCP, Capacity Engine, Dropbox Connector	ED-2, PR-3, PR-4	Coordination 70% → 40%
CD Agent	Creative Director	Onboarding, Pre-Pro, Post-Pro, Publish	Notion MCP, YouTube Connector, Retention Scorer, Competitor Scraper, Packaging Roaster	CD-1 through CD-9	Preproduction 29% → 50%
PPM Agent	Post-Production Manager	Onboarding/ Pre-Pro Overlap, Post-Pro	Frame.io Connector, QC Checker, Sentiment Analyzer, ClickUp MCP	ED-1, ED-3, ED-5	Reviews 28% → 35%, Editor Mgmt 11% → 30%
Editor Support Agent	Editor (support role)	Post-Pro	ClickUp MCP, Descript Trigger, Asset Locator	ED-3, ED-4	Core Editing 66% → 80%, Idle 6.5% → 2%

5.2 Orchestrator Agent (Meta-Agent)

The Orchestrator Agent sits above the 5 role agents. It receives all incoming requests and routes them to the correct role agent based on intent classification.

```
orchestrator_agent = Agent(
    name="DFM Orchestrator",
    instructions="""You are the routing agent for Dragonfruit Media.
    Classify the user's intent and delegate to the appropriate role agent:

    - Task creation, timelines, client management → Producer Agent
    - Task assignment, status updates, publishing prep → PC Agent
    - Ideation, scripting, research, thumbnails → CD Agent
    - QC review, editor management, feedback routing → PPM Agent
    - Editing workflow, footage ingest, asset lookup → Editor Support Agent

    If the request spans multiple roles, start with the primary role
    and use handoffs for secondary actions.
    """,
    handoffs=[producer_agent, pc_agent, cd_agent, ppm_agent, editor_agent]
)
```

6. Agent Detail: Producer Agent

> **Time Audit Context:** Producers spend 74.5% on coordination overhead (ClickUp 38% + Slack 17% + Meetings 20%). Dream target: 41%. This agent eliminates 33.5 percentage points of overhead.

6.1 Agent Configuration

Property	Value
Name	Producer Agent
Model	gpt-4o (reasoning + tool calling)
Pipeline Phases	Onboarding, Pre-Production, Post-Production (coordination)
Wish List Tools	PR-1 (Timeline Calculator), PR-2 (Client Sentiment Tracker), PR-3 (Slack Templates), PR-4 (Claude-to-ClickUp Agent)
MCP Servers	ClickUp MCP (official), Slack MCP (official)
Function Tools	timeline_calculator, client_sentiment_tracker, slack_template_library
Handoffs	→ PC Agent (task assignment), → CD Agent (creative brief)
HITL Gates	Task creation, status update, message send, assignment

6.2 Tool Definitions

PR-4: Claude-to-ClickUp Agent (via ClickUp MCP)

```
@function_tool
async def create_pod_batch(
    pod_name: str,           # e.g. "Mango Pod"
    client_name: str,        # e.g. "Client Alpha"
    content_type: str,       # "longform" | "shortform"
    num_deliverables: int,   # e.g. 4
    due_date: str,           # e.g. "2026-03-21"
    editors: list[str],      # e.g. ["Editor 1", "Editor 2"]
) -> dict:
    """Creates a full batch of ClickUp tasks for a pod.
    Uses ClickUp MCP to: list_lists → create_task (×N) → set_assignees →
set_due_dates.
    HITL gate: Presents task summary in Slack before creating.

    Maps to: PR-4 (Claude-to-ClickUp Agent)
```

```
    Dream impact: ClickUp Administration 38% → 18% (Producers)
    Pipeline steps automated: Task/Batch Creation (9.7%), General Maintenance
(7.8%)
    """
    # Step 1: Get pod's ClickUp space and lists
    # Step 2: Generate task names using DFM convention
    # Step 3: HITL gate → show summary in Slack
    # Step 4: On approval → create tasks via ClickUp MCP
    # Step 5: Confirm creation in Slack with task links
```

PR-1: Timeline Calculator

```
@function_tool
async def calculate_timeline(
    content_type: str,       # "longform" | "shortform"
    num_deliverables: int,
    complexity_tier: str,    # "standard" | "complex" | "premium"
    client_review_days: int, # Expected client turnaround
    pod_capacity: float,     # Current pod utilization (0-1)
) -> dict:
    """Computes project timeline with milestone dates and buffer windows.

    Maps to: PR-1 (Timeline Calculator)
    Dream impact: Freed time → Production Strategy & QC (3.4% → 12%)
    Output: {milestones: [...], buffer_days: int, risk_flags: [...]}
    """
```

PR-2: Client Sentiment Tracker

```
@function_tool
async def analyze_client_sentiment(
    client_name: str,
    channels: list[str],     # Slack channels to analyze
    timeframe_days: int = 14,
) -> dict:
    """Scrapes client-facing Slack messages and Frame.io comments.
    Builds running sentiment profile. Flags tone shifts and escalation risks.

    Maps to: PR-2 (Client Psychology / Sentiment Tracker)
    Integration: Slack MCP (read messages) + Frame.io API (get comments)
    Dream impact: Freed time → Client Relationship & Strategy (+10%)
    Output: {overall_sentiment: float, trend: str, risk_flags: [...],
recommendations: [...]}
    """
```

6.3 Agent ReAct Flow Example

User Input (via Slack): "Create a new batch for Mango Pod: 4 longform videos for Client Alpha, due March 21, assign to Editor 1 and Editor 2"

▼

Step 1 — REASON: Agent parses intent → batch creation. Identifies: pod=Mango, client=Alpha, type=longform, count=4, due=March 21, editors=[Editor 1, Editor 2].

▼

Step 2 — ACT: Agent calls `clickup_mcp.list_lists(space="Mango Pod")` → gets list IDs for "Longform" and "Active Batches".

▼

Step 3 — REASON: Agent generates 4 task names: "Client Alpha - LF - Batch 12 - V1 (1/4)" through "(4/4)". Assigns 2 to Editor 1, 2 to Editor 2.

▼

Step 4 — HITL GATE: Agent posts to #producer-agent-approvals in Slack:
"📋 **Batch Creation Request**
Pod: Mango | Client: Alpha | Type: Longform | Count: 4
Due: March 21 | Editors: Editor 1 (×2), Editor 2 (×2)
[✅ Approve] [❌ Reject] [✏️ Edit]"

▼

Step 5 — ACT (on approval): Agent calls `clickup_mcp.create_task()` ×4 with all fields, tags, dependencies, and due dates.

▼

Step 6 — OBSERVE + RESPOND: Agent confirms in Slack: "✅ 4 tasks created in Mango Pod. Links: [task1] [task2] [task3] [task4]. Editors notified."

6.4 Survey Context

> **From AI Tool Survey:** Producers — 4 respondents. ChatGPT 50%, Claude 50%. Use cases: Strategic Planning, Client Research, Proofreading. Top requests: Claude (2), ChatGPT Paid (1). *"I use ChatGPT primarily as a strategic thinking partner... managing multiple clients."*
>
> **Insight:** Producers already lean on Claude for strategic work. The Producer Agent extends this by connecting Claude's reasoning to ClickUp and Slack actions — turning strategic thinking into automated execution.

7. Agent Detail: Production Coordinator (PC) Agent

> **Time Audit Context:** PCs are the heaviest admin role. ClickUp 40% + Slack 11% + Meetings 19% = 70% coordination overhead. Dream target: 40%. PCs represent the highest automation ROI across all roles — most repetitive, rule-based work.

7.1 Agent Configuration

Property	Value
Name	PC Agent
Model	gpt-4o
Pipeline Phases	Pre-Production, Post-Production, Publish & Reporting
Wish List Tools	ED-2 (Capacity-Based Assignment), PR-3 (Slack Templates), PR-4 (shared with Producer)
MCP Servers	ClickUp MCP, Slack MCP
Function Tools	capacity_based_assigner, slack_template_sender, dropbox_file_locator, youtube_publish_prep
Handoffs	→ PPM Agent (QC routing), → Producer Agent (escalation)
HITL Gates	Task assignment, status update, publish action

7.2 Key Tool: ED-2 — Capacity-Based Task Assignment

```
@function_tool
async def assign_task_by_capacity(
    task_id: str,            # ClickUp task ID
    content_type: str,       # "longform" | "shortform"
    complexity: str,         # "standard" | "complex"
    pod_name: str,           # e.g. "Mango Pod"
) -> dict:
    """Automatically assigns editing tasks to editors with available capacity.

    Factors: current workload (from ClickUp), estimated duration,
    editor skill match (LF vs SF specialization), pod assignment.

    Maps to: ED-2 (Automated Capacity-Based Task Assignment)
    Dream impact: ClickUp Administration 40% → 20% (PCs)
    Pipeline steps automated: Assignees (1.6%), Task/Batch Creation (6.3%)
```

```
    Integration: ClickUp MCP (get tasks, get members) → capacity algorithm → HITL → assign
    Output: {recommended_editor: str, capacity_score: float, alternatives: [...]}
    """
    # Step 1: Query ClickUp MCP for all active tasks in pod
    # Step 2: Calculate each editor's current load (hours committed)
    # Step 3: Match task requirements to editor skills
    # Step 4: Rank editors by available capacity × skill match
    # Step 5: HITL gate → present recommendation in Slack
    # Step 6: On approval → assign via ClickUp MCP
```

7.3 Agent ReAct Flow: Slack-to-ClickUp Routing

Trigger: Client posts in #mango-pod-external Slack channel: "Hey team, we just finished shooting Episode 5. Footage is uploading to Dropbox now. Can we get a rough cut by Thursday?"

▼

Step 1 — REASON: PC Agent detects: client message, footage ready, rough cut request, deadline=Thursday. Classifies as: new task creation + assignment needed.

▼

Step 2 — ACT: Calls `capacity_based_assigner(content_type="longform", pod="Mango")` → recommends Editor 3 (lowest current load, LF specialist).

▼

Step 3 — HITL GATE: Posts to #pc-agent-approvals: "📋 New task from client message. Create 'Client Alpha - LF - Ep5 - Rough Cut' in Mango Pod? Assign to Editor 3 (capacity: 72%)? Due: Thursday. [✅ Approve] [❌ Reject]"

▼

Step 4 — ACT (on approval): Creates ClickUp task via MCP. Sends Slack notification to Editor 3. Updates client in external channel: "Got it! Rough cut assigned, targeting Thursday delivery."

7.4 Survey Context

From AI Tool Survey: PCs — 9 respondents. ChatGPT 100%, Gemini 56%. Use cases: Client Communication, Captions, Reports, Task Management, Proofreading. *"Anything that can cut down task time is good by me."*

Insight: PCs use AI exclusively for writing/communication. The PC Agent extends this by automating the task management and routing work that consumes 40% of their time — the work they can't currently offload to ChatGPT.

8. Agent Detail: Creative Director (CD) Agent

> **Time Audit Context:** CDs currently spend only 29% on preproduction (ideation + research + scripting). Dream target: 50%. The CD Agent eliminates non-creative overhead (ClickUp → 0%, non-creative meetings → 0%, post-pub monitoring → 0%) and supercharges creative work with AI tools.

8.1 Agent Configuration

Property	Value
Name	CD Agent
Model	gpt-4o (with Claude Projects for client knowledge bases)
Pipeline Phases	Onboarding, Onboarding/Pre-Pro Overlap, Pre-Production, Post-Production, Publish
Wish List Tools	CD-1 (Retention Scoring), CD-2 (Competitor Scraper), CD-3 (Packaging Roaster), CD-4 (Knowledge Bases), CD-5 (Voice Matching), CD-6 (Ideation), CD-7 (Script Drafting), CD-8 (Shorts Pulling), CD-9 (Data-to-Ideation)
MCP Servers	Notion MCP (client knowledge bases, SOPs)
Function Tools	retention_scorer, competitor_scraper, packaging_roaster, youtube_analytics, shorts_identifier, voice_matcher
Handoffs	→ PPM Agent (review routing), → Producer Agent (timeline check)
HITL Gates	Script publish, competitor report distribution, thumbnail brief send

8.2 Key Tools

CD-1: Predictive Retention Scoring Tool

```
@function_tool
async def score_retention(
    script_text: str,         # Full script
    channel_context: str,     # Client's channel name/niche
    video_format: str,        # "longform" | "shortform"
    outlier_data: str = "",   # Optional: past outlier scripts for calibration
) -> dict:
    """Predicts audience retention curve from script text.
    Identifies drop-off points, flags low-engagement sections,
    suggests structural edits including opening hook.

    Maps to: CD-1 (Predictive Retention Scoring Tool) — HIGH priority
```

```
    Dream impact: Preproduction 29% → 50% (CDs). 30% faster time-to-first-draft.
    Integration: Standalone AI model (fine-tuned on YouTube retention data)

    Output: {
        retention_curve: [{timestamp: str, predicted_retention: float}],
        drop_off_risks: [{section: str, risk_level: str, suggestion: str}],
        hook_score: float,
        hook_suggestions: [str],
        overall_score: float
    }
    """
```

CD-2: Competitor Script Scraper + Pattern Tracker

```
@function_tool
async def scrape_competitors(
    competitor_channels: list[str],  # ~20 channel URLs per client
    timeframe_days: int = 7,
    client_name: str = "",
) -> dict:
    """Monitors competitor channels, scrapes transcripts, analyzes patterns.
    Distinguishes baseline vs. genuine outliers.

    Maps to: CD-2 (Competitor Script Scrapper) — HIGH priority [feeds into CD-1]
    Integration: YouTube Data API v3 (get videos, get captions) + AI analysis
    Dream impact: Research 9% → 14% (CDs). AI accelerates research with scraping.
    Delivery: Weekly digest via Slack (automated)

    Output: {
        outlier_videos: [{title, url, views_vs_avg, hook_analysis,
narrative_structure}],
        patterns: {themes: [...], formats: [...], timing: [...]},
        digest_markdown: str  # Ready to post in Slack
    }
    """
```

CD-3: Packaging Roaster

```
@function_tool
async def roast_packaging(
    title: str,
    thumbnail_url: str = "",
    script_excerpt: str = "",  # First 500 words for hook analysis
    client_channel: str = "",
) -> dict:
    """Analyzes thumbnail + title packaging. Scores composition, color,
    facial expression, text overlay, visual tension/curiosity.
    Cross-references with Google Trends/SEO for search demand.
    Can function as Slack command: post title + thumbnail → get score + critique.

    Maps to: CD-3 (Packaging Roaster) — HIGH priority
    Integration: Vision model (thumbnail analysis) + Google Trends API + YouTube
API
```

```
    Dream impact: Ideation 11% → 24% (CDs). Thumbnail AI tools integrated.

    Output: {
        overall_score: float,
        composition_analysis: str,
        curiosity_gap_score: float,
        suggested_title_variants: [str],
        thumbnail_critique: str,
        seo_validation: {search_volume: int, trend: str}
    }
    """
```

CD-4: Client-Specific Knowledge Bases (via Notion MCP)

```
@function_tool
async def query_client_knowledge_base(
    client_name: str,
    query: str,              # Natural language question
) -> dict:
    """Queries a client-specific knowledge base pre-loaded with:
    brand voice docs, past scripts, performance data, audience demographics, style
guides.

    Maps to: CD-4 (Client-Specific Knowledge Bases) — HIGH priority
    Integration: Notion MCP (get_page, search) + RAG over client documents
    Dream impact: Scripting 5% → 12% (CDs). On-brand suggestions, reference past
decisions.

    Implementation: Claude Projects (one per client) OR Notion-integrated RAG bot.
    Each project pre-loaded with client's Notion workspace content.
    """
```

8.3 Survey Context

> **From AI Tool Survey:** CDs — 5 respondents. ChatGPT 100%, Firefly 40%, Gemini 40%. Use cases: Ideation, Scripting, Thumbnail Design, Client Comms. *"I haven't sat down to learn the other tools despite knowing they're highly valuable."* — George A., CD
>
> **Insight:** CDs have the biggest knowledge gap. All 5 use ChatGPT but haven't explored beyond it. The CD Agent brings the tools TO them — they don't need to learn 9 new tools. They interact with one agent that orchestrates CD-1 through CD-9 behind the scenes. Training impact: outsized.

9. Agent Detail: Post-Production Manager (PPM) Agent

> **Time Audit Context:** PPMs are overloaded — logged 242.5h against 200h capacity (121%). Current split: Reviews 28%, Style Guides 19%, Meetings 25%, Slack 14%. Dream: Reviews 35%, Editor Management 30%. The PPM Agent eliminates reactive overhead so PPMs can focus on intentional quality coaching.

9.1 Agent Configuration

Property	Value
Name	PPM Agent
Model	gpt-4o
Pipeline Phases	Onboarding/Pre-Pro Overlap, Post-Production
Wish List Tools	ED-1 (Automated QC Checker), ED-3 (Rough Cut Acceleration), ED-5 (Frame.io Sentiment Analyzer)
MCP Servers	ClickUp MCP (task routing), Slack MCP (notifications)
Function Tools	video_qc_checker, frameio_sentiment_analyzer, editor_performance_tracker, rough_cut_evaluator
Handoffs	→ CD Agent (creative escalation), → PC Agent (assignment changes)
HITL Gates	QC report distribution, editor feedback send, escalation trigger

9.2 Key Tools

ED-1: Automated Video QC Checker

```
@function_tool
async def run_qc_check(
    video_url: str,          # Frame.io asset URL or local path
    client_name: str,        # For client-specific checklist
    content_type: str,       # "longform" | "shortform"
) -> dict:
    """Reviews rendered video against client-specific QC checklist.
```